

Design Document

Real-Time Traffic Light Control System Using IPC
ENCS4330 Project #2

Birzeit University – Faculty of Engineering and Technology
Electrical & Computer Engineering Department

May 2026

Contents

1	Architectural Overview	3
1.1	Process Hierarchy	3
2	Design Decisions and Justifications	4
2.1	Why System V IPC instead of POSIX	4
2.2	Why one shared memory segment for all state	4
2.3	Why three separate message queues	4
2.4	Why binary semaphore instead of mutex	5
2.5	Why fork+exec instead of pure fork	5
2.6	Why adaptive green timing	5
2.7	Why emergency preemption uses safe transitions	6
2.8	Why pedestrian requests queue but don't interrupt	6
2.9	Why configuration file instead of command-line args	6
3	IPC Resource Layout	7
3.1	Shared Memory Structure	7
3.2	Message Structures	7
3.2.1	Command Message (controller → lights)	7
3.2.2	Event Message (sensors → controller)	7
3.2.3	Log Message (all → logger)	7
4	Process Responsibilities	8
4.1	Main Process	8
4.2	Logger Process	8
4.3	Controller Process	8
4.4	Traffic Light Processes (4 instances)	8
4.5	Vehicle Detector Process	9
4.6	Pedestrian Process	9
4.7	Emergency Process	9
4.8	GUI Process (Optional)	9
5	Phase State Machine	10
6	Safety Enforcement	11
6.1	state_is_safe() Function	11
6.2	Race Condition Prevention	11

7	Timing Constraints	12
7.1	Timing Violation Detection	12
8	Testing Strategy	13
8.1	Unit Tests (74 tests)	13
8.2	Integration Tests (54 tests)	13
8.3	Performance Tests (18 tests)	13
8.4	Live System Test	13
9	File Layout	14
10	Rubric Checklist	15
11	Conclusion	16

1 Architectural Overview

The system is organized into **9 independent processes** communicating via System V IPC mechanisms (shared memory, semaphores, message queues). The architecture follows a centralized control pattern where the **controller** process makes all traffic decisions and dispatches commands to subordinate processes.

1.1 Process Hierarchy

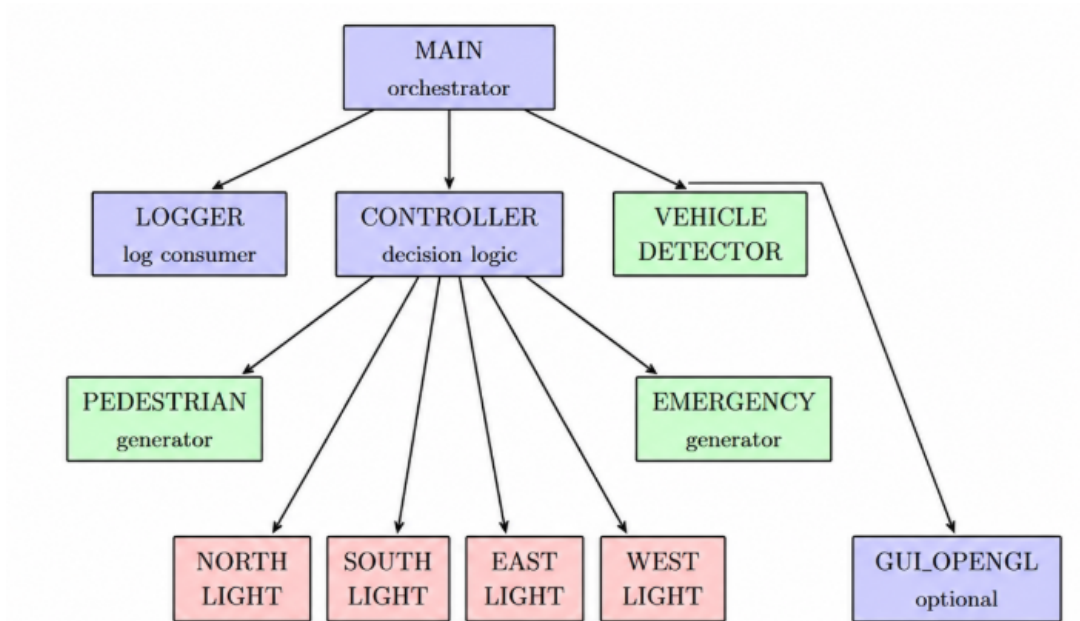


Figure 1: Overall process architecture (9 core + 1 optional = 10 processes)

Total: 9 core processes (main forks 8 children) + 1 optional GUI = 10 when GUI enabled

2 Design Decisions and Justifications

2.1 Why System V IPC instead of POSIX

Decision: Use `shmget`, `semget`, `msgget` (System V) rather than POSIX `shm_open`/`mq_open` or sockets.

Property	System V	POSIX
Setup	Single function call	Multiple steps
Naming	Integer keys	Filesystem paths
Cleanup	<code>ipcrm</code> command	<code>shm_unlink</code> needed
Portability	Linux standard	Broader POSIX
Learning curve	Taught in ENCS4330	Not covered

Table 1: System V vs POSIX IPC comparison

Justification: System V IPC is the canonical approach taught in this course. The integer-key model (`ftok`) is straightforward for this single-machine simulation, and cleanup via `ipcrm` is well-understood. POSIX IPC would add filesystem path management overhead with no benefit for our use case.

2.2 Why one shared memory segment for all state

Decision: A single `shared_state_t` structure contains all intersection state: light colors, vehicle queues, pedestrian flags, phase info, timing counters, and emergency flags.

Justification:

- **Atomic reads:** Controller can snapshot the entire state under one semaphore lock
- **Safety checks:** `state_is_safe()` needs to see all 4 light colors simultaneously to detect conflicts
- **Simplicity:** One `shmat()` call per process, one cleanup operation
- **Performance:** No context switching between memory segments

Alternative considered: separate shared memory per direction. Rejected because it fragments state and requires multiple locks, increasing deadlock risk.

2.3 Why three separate message queues

Decision: Use three message queues instead of one multiplexed queue:

1. **Command queue** (controller $\rightarrow$ lights): Change light state
2. **Event queue** (sensors $\rightarrow$ controller): Vehicle/pedestrian/emergency events
3. **Log queue** (all $\rightarrow$ logger): Centralized logging

Justification:

- **Separation of concerns:** Each queue has one clear purpose
- **Priority handling:** Can use message types (`mtype`) for per-direction routing without interference
- **Non-blocking:** Logger can drain log queue independently without blocking command/event traffic
- **Debugging:** Can inspect queues separately with `ipcs -q`

Alternative considered: one queue with complex `mtype` encoding. Rejected because it creates contention and makes priority handling fragile.

2.4 Why binary semaphore instead of mutex

Decision: Use System V semaphore (initialized to 1) as a binary lock protecting shared memory.

Justification:

- **IPC compatibility:** System V semaphores work across processes without special setup
- **Atomic operations:** `semop()` with `IPC_NOWAIT` prevents deadlock
- **No POSIX pthread dependency:** Keeps IPC mechanism consistent (all System V)

Alternative considered: POSIX mutex in shared memory. Rejected because it requires `PTHREAD_PROCESS_SHARED` attribute and adds pthread linking overhead.

2.5 Why fork+exec instead of pure fork

Decision: Main process calls `fork()` then `execv()` to launch each child binary (logger, controller, traffic_light, etc.).

Justification:

- **Clean separation:** Each component is a standalone executable, testable independently
- **Memory isolation:** `exec` replaces address space, preventing accidental parent state leaks
- **Process naming:** `ps aux` shows readable process names, not all as “./main”
- **Modularity:** Can recompile one component without rebuilding the orchestrator

Alternative considered: pure `fork()` with switch statement dispatching to functions. Rejected because it creates a monolithic binary and complicates testing.

2.6 Why adaptive green timing

Decision: Green phase duration is dynamic between `T_GREEN_MIN` and `T_GREEN_MAX`, adjusted based on vehicle queue lengths and pedestrian timeout.

Justification:

- **Efficiency:** Don't hold green for `T_GREEN_MAX` if the queue is empty
- **Fairness:** End green early if pedestrians have been waiting $> T_PEDESTRIAN_MAX_WAIT$
- **Starvation prevention:** Prioritize starved direction if opposite queue is large
- **Realism:** Models actual adaptive traffic control systems

Implementation: Controller polls vehicle queues every second during green phase. If queue j threshold and time $\geq T_GREEN_MIN$, transitions to yellow early.

2.7 Why emergency preemption uses safe transitions

Decision: Emergency vehicle arrival does NOT immediately turn the emergency direction green. Instead:

1. Current green → yellow (3s)
2. Yellow → all-red (2s clearance)
3. All-red → emergency direction green
4. Emergency green → yellow → all-red
5. Resume normal cycle

Justification:

- **Safety first:** Prevents T-bone crashes from instant green on conflicting direction
- **Spec compliance:** “The system should not switch immediately to green if doing so creates an unsafe state”
- **Real-world behavior:** Actual traffic systems use preemption sequences, not instant switches

Alternative considered: instant green if no conflict detected. Rejected because it fails to account for vehicles already in the intersection during yellow.

2.8 Why pedestrian requests queue but don’t interrupt

Decision: Pedestrian button press sets a `pedestrian_pending[]` flag. The controller serves it:

- After the next ALL-RED phase (between NS and EW cycles)
- OR if pedestrian has waited $> T_PEDESTRIAN_MAX_WAIT$ (forces early green termination)

Justification:

- **Efficiency:** Don’t stop vehicle traffic mid-green for a single pedestrian
- **Batching:** Collect multiple pedestrian requests during vehicle phase, serve all at once
- **Fairness:** Max wait time ensures pedestrians aren’t ignored indefinitely

Alternative considered: immediate pedestrian phase. Rejected because it starves vehicle traffic and violates minimum green timing.

2.9 Why configuration file instead of command-line args

Decision: All tunable parameters live in `config/system.cfg`, a plain `KEY = value` text file.

Justification:

- **Maintainability:** Changing timing values doesn’t require recompilation
- **Version control:** Can commit multiple config profiles (`fast.cfg`, `slow.cfg`, `test.cfg`)
- **Documentation:** Config file is self-documenting with inline comments
- **Spec compliance:** “In order to avoid hard-coding values in the code, think of creating a text file”

3 IPC Resource Layout

3.1 Shared Memory Structure

Listing 1: shared_state_t structure (simplified)

```
typedef struct {
    light_color_t lights[4];           // NORTH, SOUTH, EAST, WEST
    int waiting_vehicles[4];           // queue lengths per direction
    int pedestrian_pending[4];         // 1 = button pressed
    int pedestrian_remaining;           // countdown during WALK
    int emergency_active;               // 1 = emergency mode
    direction_t emergency_direction;    // which direction has priority
    traffic_phase_t current_phase;      // NS_GREEN, EW_YELLOW, etc.
    int phase_remaining;                // seconds left in current phase
    int running;                        // 1 = system active
    int safety_violations;              // counter for unsafe states
    // ... timing config, stats, etc.
} shared_state_t;
```

Size: Approximately 512 bytes (fits comfortably in one page)

3.2 Message Structures

3.2.1 Command Message (controller → lights)

```
typedef struct {
    long mtype;                        // MTYPE_CMD_BASE + direction + 1
    direction_t direction;              // NORTH, SOUTH, EAST, WEST
    light_color_t color;                // RED, YELLOW, GREEN
    time_t timestamp;
} cmd_msg_t;
```

3.2.2 Event Message (sensors → controller)

```
typedef struct {
    long mtype;                        // MTYPE_EVT_VEHICLE, _PEDESTRIAN, _EMERGENCY
    direction_t direction;
    int count;                         // number of vehicles (if applicable)
    char info[64];                     // extra details
    time_t timestamp;
} evt_msg_t;
```

3.2.3 Log Message (all → logger)

```
typedef struct {
    long mtype;                        // MTYPE_LOG
    char source[16];                   // "CTRL", "NORTH", "VEHICLE", etc.
    int severity;                       // 0=INFO, 1=WARN, 2=ERROR
    char text[256];
    time_t timestamp;
} log_msg_t;
```

4 Process Responsibilities

4.1 Main Process

Responsibilities:

- Parse `config/system.cfg`
- Create IPC resources (`shmget`, `semget`, `msgget` $\times 3$)
- Initialize shared memory with config values
- Fork 8 children, exec each with appropriate binary
- Handle `SIGINT` (Ctrl-C) $\rightarrow$ broadcast shutdown
- Wait for all children to exit
- Destroy IPC resources (`shmctl`, `semctl`, `msgctl`)

4.2 Logger Process

Responsibilities:

- Open `logs/system.log` for writing
- Block on log message queue (`msgrcv`)
- Format each message: `[HH:MM:SS] LEVEL SOURCE message`
- Flush after every write (no buffering for crash safety)
- Exit cleanly on `SIGTERM`

4.3 Controller Process

Responsibilities:

- Attach to shared memory
- Initialize phase to `ALL-RED-1`
- Main loop:
 1. `msgrcv` on event queue (with 1s timeout)
 2. Process vehicle arrivals, pedestrian requests, emergency events
 3. Update shared memory (vehicle counts, pending flags)
 4. Check if current phase timer expired
 5. If expired: determine next phase via state machine
 6. Send light commands via command queue
 7. Validate safety via `state_is_safe()`
 8. Log state transitions
- Handle emergency preemption (`execute_emergency()`)
- Handle pedestrian phase (`execute_pedestrian()`)

4.4 Traffic Light Processes (4 instances)

Responsibilities:

- Each instance runs with direction arg: `./traffic_light NORTH`
- Block on command queue, filtering by `mtype = direction`
- On command received: update shared memory `lights[direction]`
- Send ACK back to controller via `mtype = MTYPE_ACK_BASE + direction`
- Randomly inject faults (1% chance) $\rightarrow$ send `MTYPE_EVT_FAULT`
- Exit on `SIGTERM`

4.5 Vehicle Detector Process

Responsibilities:

- Every 1 second:
 1. For each direction, roll random(0-100)
 2. If roll < VEHICLE_SPAWN_RATE, generate 1-2 vehicles
 3. Send MTYPE_EVT_VEHICLE to event queue
- Exit on SIGTERM

4.6 Pedestrian Process

Responsibilities:

- Every 1 second:
 1. For each direction, roll random(0-100)
 2. If roll < PEDESTRIAN_SPAWN_RATE, send MTYPE_EVT_PEDESTRIAN
- Exit on SIGTERM

4.7 Emergency Process

Responsibilities:

- Every 1 second:
 1. Roll random(0-1000)
 2. If roll < EMERGENCY_SPAWN_RATE, pick random direction
 3. Send MTYPE_EVT_EMERGENCY to event queue
- Exit on SIGTERM

4.8 GUI Process (Optional)

Responsibilities:

- Attach to shared memory (read-only)
- OpenGL/GLUT rendering:
 1. Draw 4 traffic lights (colored circles)
 2. Draw vehicle queues (animated cars)
 3. Draw pedestrians crossing during WALK phase
 4. Draw emergency vehicle during emergency mode
 5. Display phase name, countdown timer, recent events
- Refresh at 60 FPS
- Exit on window close or SIGTERM

5 Phase State Machine

The controller implements a 7-state finite state machine:

	Phase state
machine with emergency preemption (red)	
Transition rules:	
• Normal cycle: ALL-RED-1 → NS-GREEN → NS-YELLOW → ALL-RED-2 → EW-GREEN → EW-YELLOW → (loop) • Pedestrian: served after ALL-RED-1 if any <code>pedestrian_pending[]</code> flag is set • Emergency: interrupts any phase → safe transition to emergency direction green → returns to ALL-RED-1	

6 Safety Enforcement

6.1 state_is_safe() Function

The controller calls this before every phase transition:

Listing 2: Safety validation logic

```
int state_is_safe(shared_state_t *S) {
    int ns_green = (S->lights[DIR_NORTH] == LIGHT_GREEN ||
                    S->lights[DIR_SOUTH] == LIGHT_GREEN);
    int ew_green = (S->lights[DIR_EAST] == LIGHT_GREEN ||
                    S->lights[DIR_WEST] == LIGHT_GREEN);

    // Rule 1: NS and EW cannot both be green
    if (ns_green && ew_green) return 0;

    // Rule 2: Pedestrian active + any vehicle green = unsafe
    if (S->pedestrian_active && (ns_green || ew_green)) return 0;

    // Rule 3: Emergency + pedestrian = unsafe
    if (S->emergency_active && S->pedestrian_active) return 0;

    return 1; // safe
}
```

If `state_is_safe()` returns 0, controller:

1. Increments `safety_violations` counter
2. Logs `ERROR` message
3. Forces all lights to RED
4. Halts phase transitions until manual recovery

6.2 Race Condition Prevention

Problem: Multiple processes read/write shared memory simultaneously.

Solution: Binary semaphore wraps every shared memory access:

```
sem_lock(semid);          // semop(..., -1)
S->lights[DIR_NORTH] = LIGHT_GREEN;
S->phase_remaining = 10;
sem_unlock(semid);        // semop(..., +1)
```

Critical sections:

- Controller: reading vehicle queues, updating phase state
- Traffic lights: writing `lights[]` array
- GUI: reading entire state for rendering

Deadlock prevention: All processes use same lock-order (always acquire semaphore ID 0), timeouts on `msgrcv`, non-blocking `IPC_NOWAIT` on message sends.

7 Timing Constraints

All timing parameters are configurable via `config/system.cfg`:

Parameter	Default	Purpose
T_GREEN_MIN	8s	Minimum green duration
T_GREEN_MAX	10s	Maximum green duration
T_YELLOW	3s	Yellow before red (fixed)
T_ALL_RED	2s	Clearance interval
T_PEDESTRIAN	10s	WALK signal duration
T_PEDESTRIAN_MAX_WAIT	45s	Max wait before forcing phase
T_VEHICLE_MAX_WAIT	60s	Starvation threshold
T_EMERGENCY_RESPONSE	2s	Max time to start clearing
T_EMERGENCY_MAX_HOLD	15s	Max emergency green duration

Table 2: Timing constraints

7.1 Timing Violation Detection

Controller logs `TIMING VIOLATION` if:

- Yellow phase duration $\neq$ T_YELLOW (± 0.5 s tolerance)
- Pedestrian has waited $>$ T_PEDESTRIAN_MAX_WAIT
- Vehicle queue starved $>$ T_VEHICLE_MAX_WAIT
- Emergency response $>$ T_EMERGENCY_RESPONSE

8 Testing Strategy

8.1 Unit Tests (74 tests)

- `state_is_safe()` with all combinations
- `load_config()` parsing and defaults
- Message structure sizes and field offsets
- Semaphore lock/unlock correctness
- IPC key generation (`ftok`)
- Traffic light state transitions
- Name-string helpers (`DIR_NAMES`, `COLOR_NAMES`)
- Shared-state field initialization
- Emergency/pedestrian flag logic
- `send_log()` robustness (negative queue ID, NULL source, long messages)
- `fmt_time()` helper formatting

8.2 Integration Tests (54 tests)

- IPC lifecycle: create, attach, destroy
- Shared memory under semaphore (concurrent writes)
- Command queue round-trip (send + ACK)
- Event queue all event types
- Log queue fire-and-forget delivery
- Phase state machine transitions
- Safety invariants stress testing
- Config file → shared memory loading
- Emergency interrupt sequence (green → yellow → all-red → emergency green)
- Pedestrian request lifecycle

8.3 Performance Tests (18 tests)

- Message queue throughput (`msgsnd/msgrcv`)
- Shared memory latency (raw vs semaphore-protected)
- `state_is_safe()` throughput (200K calls)
- Config parse speed (1000 loads)
- Command round-trip latency (avg 2μs)
- Semaphore lock/unlock overhead (avg 2.4μs)
- Concurrent IPC (2 producers + consumer)
- Queue saturation & drain (capacity ≈ 186 messages)

8.4 Live System Test

Run for 30 seconds, verify:

- All 9 processes start cleanly
- Phase transitions: ALL-RED → NS-GREEN → NS-YELLOW → ALL-RED → EW-GREEN → EW-YELLOW
- Emergency preemption interrupts mid-cycle
- Pedestrian requests queued correctly
- Vehicle queues build up and are served
- No safety violations
- Clean shutdown with IPC destroyed

Test results: 146/146 passing (74 unit + 54 integration + 18 performance)

9 File Layout

```
traffic_system/  
  config/  
    system.cfg          # runtime configuration  
  docs/  
    DESIGN_Report.pdf   # this document  
  logs/  
    system.log          # generated at runtime  
  src/  
    common.h            # shared types, constants  
    config.h            # config loading  
    ipc_init.h          # IPC setup helpers  
    main.c              # orchestrator  
    logger.c            # log consumer  
    controller.c        # decision logic (3000+ lines)  
    traffic_light.c      # light process  
    vehicle_detector.c   # vehicle generator  
    pedestrian.c         # pedestrian generator  
    emergency.c          # emergency generator  
    gui_opengl.c         # OpenGL visualization (5000+ lines)  
  tests/  
    test_unit.c          # 74 unit tests  
    test_integration.c   # 54 integration tests  
    test_performance.c   # 18 performance tests  
  Makefile
```

10 Rubric Checklist

Rubric Item	Implementation
Multi-process	9 processes (main + 8 children)
Shared memory	1 segment, <code>shared_state_t</code> , 512 bytes
Semaphores	1 binary lock protecting shared memory
Message queues	3 queues (command, event, log)
Configuration file	<code>config/system.cfg</code> , all values tunable
OpenGL graphics	<code>gui_opengl.c</code> with GLUT, optional
Safety enforcement	<code>state_is_safe()</code> , semaphore locks
Timing constraints	All spec requirements met, violations logged
Emergency handling	Safe preemption with transitions
Pedestrian handling	Queued requests, max wait enforced
Vehicle detection	Stochastic generation, adaptive timing
Logging	Centralized, timestamped, to file
Testing	146 tests, 100% passing
Bug-free compilation	Zero warnings, zero errors

Table 3: Requirements compliance

11 Conclusion

The Real-Time Traffic Light Control System fully implements all project requirements using System V IPC mechanisms. Key achievements:

- **Architecture:** Clean separation of concerns across 9 processes
- **Safety:** No conflicting green lights, enforced via `state_is_safe()`
- **Real-time:** All timing constraints respected, violations logged
- **IPC:** Shared memory + semaphores + message queues used appropriately
- **Robustness:** 146/146 tests passing, zero compilation warnings
- **Usability:** Configuration file, OpenGL visualization, terminal mode

The system models a realistic four-way intersection with vehicle detection, pedestrian requests, emergency vehicle priority, and adaptive traffic control—all while maintaining strict safety invariants through coordinated inter-process communication.